

Putting It All Together: Object-Oriented Programming Lab (CodeGrade)

 <https://github.com/learn-co-curriculum/python-p3-oop-putting-it-all-together>  <https://github.com/learn-co-curriculum/python-p3-oop-putting-it-all-together/issues/new>

Learning Goals

- Create and instantiate classes in Python.
- Build methods that perform functions tailored to their unique objects.
- Use the `property()` function to create properties and validate input.

Key Vocab

- **Class**: a bundle of data and functionality. Can be copied and modified to accomplish a wide variety of programming tasks.
- **Initialize**: create a working copy of a class using its `__init__` method.
- **Instance**: one specific working copy of a class. It is created when a class's `__init__` method is called.
- **Object**: the more common name for an instance. The two can usually be used interchangeably.
- **Object-Oriented Programming**: programming that is oriented around data (made mobile and changeable in **objects**) rather than functionality. Python is an object-oriented programming language.
- **Function**: a series of steps that create, transform, and move data.
- **Method**: a function that is defined inside of a class.
- **Magic Method**: a special type of method in Python that starts and ends with double underscores. These methods are called on objects under certain conditions without needing to use their names explicitly. Also called **dunder methods** (for **double underscore**).
- **Attribute**: variables that belong to an object.
- **Property**: attributes that are controlled by methods.

Introduction

Object-oriented programming, or OOP, is an extremely useful programming paradigm in which we can organize our code according to how real-world objects might interact with one another. We can wrap properties/data and behavior up in classes, and then create instances, or individual "members", of those classes that can interact with one another.

One common misconception about OOP is that everything *MUST* model the *real world*. If we limit our objects to things in the real world, the limitations will start jumping out at us.

Imagine a phone call between 2 people. Sure, the PEOPLE are real, but what about the phone call? If we think about the phone call through OOP, we can model it too! A phone call has a `caller` and a `receiver`, a `duration`, and maybe even a `cost_per_minute`. In the *real world*, it's not a real thing, but in OOP IT IS!

In this lab, you will put together everything you've learned so far about Object Orientation in Python. You will be building out two classes, a `Book` class and a `Shoe` class.

Instructions

This lab is test-driven. You will write your code in `lib/book.py` and `lib/shoe.py`. Run the tests and work your way through the test errors one by one until you get everything passing.

You're also encouraged to look at the test files to see what the tests are expecting to be able to do with your classes. These tests won't force you to use everything that you've learned in this module- feel free to add any features that might be useful!

Note that there are separate test files for the two classes inside the `testing` folder. If you'd like to run the tests separately for the two classes, you can specify which test file to run:

```
$ pytest -x testing/book_test.py
```

or:

```
$ pytest -x testing/shoe_test.py
```

Remember that the optional `-x` flag makes your tests stop after the first failure- this setting is ideal for test-driven development!

Happy coding!

This tool needs to be loaded in a new browser window

Load Putting It All Together: Object-Oriented Programming Lab (CodeGrade) in a new window